

Generalizacja (Generalisation) - Notatki z wykładu (tłumaczenie PL)

Materiał źródłowy: Dr. Brian Mc Ginley (Week 2 - Generalisation.pdf) · Tłumaczenie: Marcin

7 października 2026

Slajdy są w kolejności wykładu. Elementy graficzne, których nie da się przepisać (wykresy, zdjęcia), są opisane w nawiasach kwadratowych. Wzory matematyczne i fragmenty kodu pozostawiono w oryginalnej notacji.

1. Generalizacja

- Dr. Brian Mc Ginley

2. Generalizacja

- Generalizacja (*Generalisation*) jest celem uczenia maszynowego
 - Chcemy dobrej skuteczności dla **nowych** danych
 - O jakości generalizacji decyduje to, jak dobrze model radzi sobie z wcześniej „niewidzianymi danymi”, tzn. danymi, które nie były użyte do trenowania modelu.
- **Definicja:**
 - Generalizacja to zdolność modelu do generowania sensownych wyników dla zestawów danych wejściowych, których nigdy wcześniej nie widział.
- Do pomiaru można użyć błędu średniokwadratowego (pierwiastek z błędu średniokwadratowego, *Root-mean-squared (RMS) error*):

$$**E_RMS = \sqrt{(2E(w^*) / N)} **$$

- Wróćmy do wcześniejszego pomysłu sprawdzania, jak daleko *pred* jest od oryginalnego *y* – jaki jest z tym problem?
- Sprawdzamy generalizację modelu na danych, na których był trenowany – czyli NIE są to dane wcześniej niewidziane.
- Nasz model może być **przeuczony** (*overfit*) na danych treningowych.

3. Niedouczenie (Underfitting)

- Przeciwnieństwem jest problem niedouczenia (*under-fitting*).
- Nie da się nawet zredukować straty treningowej (*training loss*) do rozsądnego poziomu. To jak student, który nie jest wystarczająco bystry, żeby rozwiązać ćwiczenia.
- W uczeniu maszynowym jest to zazwyczaj objaw zbyt mało złożonego modelu (np. student niewystarczająco bystry).
- Student niewystarczająco bystry jest mało prawdopodobny w prawdziwym świecie, ponieważ każdy z nas dostał do pracy złożony model (ludzki mózg).
- Inne przyczyny niedouczenia w uczeniu maszynowym to słabe mechanizmy trenowania lub nieodpowiednia funkcja straty.
- To bardziej przypomina studenta, który cały dzień siedzi w bibliotece, przegląda media społecznościowe i nie może zrozumieć, dlaczego nadal nie umie rozwiązać przykładów.

4. Niedouczenie / Przeuczenie

- **Definicja:**
- Niedouczenie występuje, gdy model statystyczny lub algorytm uczenia maszynowego nie potrafi uchwycić podstawowego trendu w danych.
- Intuicyjnie: niedouczenie występuje, gdy model lub algorytm nie dopasowuje się do danych wystarczająco dobrze. Często jest wynikiem nadmiernie prostego modelu.

- **Definicja:**

- Przeuczenie (*overfitting*) występuje, gdy model statystyczny lub algorytm uczenia maszynowego uchwyci szum w danych. Intuicyjnie: przeuczenie występuje, gdy model lub algorytm dopasowuje się do danych treningowych zbyt dobrze.
- Przeuczenie jest często wynikiem nadmiernie skomplikowanego modelu.

5. Budowanie modelu

- Nie ma znaczenia, jaki typ modelu budujemy – proces jest podobny.
 - Mamy zbiór danych.
 - Wybieramy podstawową postać modelu.
 - **Trenujemy** model na danych (dobierając hiperparametry (*hyper-parameters*), aby „dostroić” model, np. jaki stopień wielomianu).
 - **Testujemy** model.
- Uwaga: bez względu na to, jak dobry jest model, prawie zawsze im więcej punktów danych, tym lepiej. (Im więcej krotek $\mathbf{x}$, y , tym lepiej; zbyt wiele zmiennych niezależnych/cech może powodować problemy.)

6. Budowanie modelu (2)

- Mamy zbiór danych będący parami informacji. Znamy poprawną odpowiedź dla tego zbioru.
 - X to zbiór wszystkich danych wejściowych (macierz).
 - y to zbiór wszystkich odpowiadających zmiennych odpowiedzi.
 - Konkretny $y^{(i)}$ jest wynikiem dla konkretnego $\mathbf{x}^{(i)}$.

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X,y)
pred = lr.predict(X)
```

7. Testowanie skuteczności modelu

```
lr.fit(X, y)
pred = lr.predict(X)
```

- W kodzie utworzyliśmy model o nazwie *lr*.
 - *lr* został **wytrenowany** na zbiorze *znanych* informacji.
 - Pamiętajmy z regresji, że wartości przewidywane nie są dokładnie takie same jak wejściowe.
 - Zatem *pred* nie jest dokładnie równe y , ale miejmy nadzieję, że są blisko.
 - Moglibyśmy więc testować skuteczność, sprawdzając, jak daleko *pred* jest od oryginalnego y .
 - Czy ktoś widzi z tym problem?

8. Przeuczenie / Niedouczenie – powtórka

- Przeuczenie to przypadek, w którym całkowity koszt jest naprawdę mały, ale generalizacja modelu jest zawodna. Wynika to z tego, że model uczy się „za dużo” z zestawu danych treningowych. (niskie obciążenie, wysoka wariancja; *low bias, high variance*)
- Niedouczenie to przypadek, w którym model „nie nauczył się wystarczająco dużo” z danych treningowych, co skutkuje niską generalizacją i zawodnymi predykcjami. (wysokie obciążenie, niska wariancja; *high bias, low variance*)

9. Obciążenie / Wariancja (Bias / Variance)

[Trzy wykresy punktowe obok siebie (oś pozioma – Time, oś pionowa – Values), na każdym te same czarne punkty tworzące kształt litery U (spadek, a potem wzrost wartości) z szumem. Lewy, „Underfitted”: punkty dopasowane prostą rosnącą linią – za prosty model. Środkowy, „Good Fit/Robust”: gładka krzywa w kształcie litery U przebiegająca przez środek punktów. Prawy, „Overfitted”: bardzo poszarpana krzywa przechodząca przez niemal każdy punkt.]

- Obciążenie (*Bias*): wysokie obciążenie (przyjmowanie wielu założeń o modelu leżącym u podstaw danych) może skutkować niedouczeniem.

- Wariancja (*Variance*): wrażliwość modelu na dane treningowe
 - Większa wrażliwość = wyższa wariancja
- Kompromis obciążenie/wariancja (*Bias/Variance trade-off*)

10. Przeuczenie / Niedouczenie - powtórka (2)

- Kompromis obciążenie-wariancja. Jaka jest właściwa miara?
 - W zależności od modelu, wynik leżący pomiędzy przeuczeniem a niedouczeniem jest bardziej pożądany.
 - Ten kompromis jest najważniejszym aspektem trenowania modeli uczenia maszynowego. Modele uczenia maszynowego spełniają swoje zadanie, gdy dobrze generalizują.
 - Generalizacja jest ograniczona przez dwa niepożądane skrajne wyniki - wysokie obciążenie i wysoką wariancję.
 - Wykrycie, czy model cierpi na jedno z nich, jest wyłączną odpowiedzialnością twórcy modelu.

11. Pewność (Confidence)

- Jednym z głównych problemów przeuczenia jest to, że model nie tylko błędnie klasyfikuje nowe dane, ale często robi to z bardzo wysoką pewnością.
- Gdy model ma bardziej umiarkowaną złożoność, niektóre próbki ze zbioru treningowego mogą znaleźć się po złej stronie granicy decyzyjnej (klasyfikacja) lub daleko od modelu (regresja).
- Model może jednak o tym poinformować.
- Model może powiedzieć: przewiduję taki wynik, ale jestem pewien tylko w x% (klasyfikacja) lub ta predykcja ma tolerancję x (regresja).

12. Jak testujemy model

- Używamy zbioru testowego, dla którego znamy odpowiedzi.
- Nie ma znaczenia, czy zmienna odpowiedzi jest liczbą (regresja), czy cechą jakościową (kategoria/klasyfikator) - idea jest podobna.
- Do określania dokładności i skuteczności możemy używać różnych wzorów.
- Nasz zbiór testowy jest trzymany oddzielnie od zbioru treningowego.
- Gdy służy do sprawdzenia skuteczności modelu, są to dla modelu zupełnie nowe informacje - a my nadal znamy poprawne odpowiedzi.

```
1 lr.fit(X_train , y_train)
2 pred = lr.predict(X_test)
3 #Compare pred with y_test
```

- Porównanie *pred* ze znanymi wynikami dla zbioru testowego pozwoli nam sprawdzić skuteczność.

13. Zbiory testowe (Test sets)

- Czasami, rozwiązując problem, dostajemy jawnie zbiór treningowy (*Training Set*) i osobny zbiór testowy (*Test Set*).
- Zbiór treningowy służy do dopasowania (*fit*) modelu (nauczenia parametrów).
- Zbiór testowy służy do oceny (*evaluate*) modelu (sprawdzenia jego skuteczności/generalizacji).
- Jednak często dostajemy tylko jeden zbiór danych i do nas należy wybór najlepszej metody jego podziału.

14. Podział na zbiór treningowy i testowy (Train/Test split)

- Załóżmy, że zaczynamy od dużego zbioru zebranych danych. Jest to próba o rozmiarze m .
 - Dla każdego elementu i w danych znamy wektor $\mathbf{x}^{(i)}$ (cechy/zmienne niezależne)
 - Znamy też odpowiadającą odpowiedź $y^{(i)}$.
 - Dzielimy dane na dwa zbiory: treningowy i testowy
 - Prawie zawsze zbiór treningowy jest znacznie większy od testowego. Podziały 80/20 i 75/25 są często dobre.
 - Losowo wybieramy, które elementy trafiają do zbioru treningowego, a które do testowego (**bardzo ważne**).

- **Dopasowujemy** model na zbiorze **treningowym**.
- **Oceniamy** model na zbiorze **testowym**.

15. Podział na zbiór treningowy i testowy (2)

```
from sklearn.model_selection import train_test_split
```

- jest wbudowaną w sklearn metodą losowego podziału zbioru danych na treningowy i testowy.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
```

- utworzy zbiór testowy o rozmiarze 25% danych (co i tak jest wartością domyślną)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.75)
```

- można to też określić w ten sposób, jeśli ktoś woli.
- Zwracane wartości to tablice numpy.

16. Pełna procedura budowy modelu

[Diagram przepływu. Po lewej cylinder „Data” ze strzałką do tabeli „Results Known” (tabela z kolumną wyników po prawej stronie: 0, 3, 2, 5, 1, pozostałe wiersze puste). Z górnych wierszy tabeli strzałka prowadzi do dokumentu „Training set” (kolor pomarańczowy), a stąd w dół do ramki „Model Builder”. Z dolnych wierszy tabeli strzałka prowadzi w dół do dokumentu „Testing set” (kolor zielony). Model Builder tworzy model (zielony trapez z drzewem decyzyjnym Y/N); zbiór testowy trafia do modelu, a model generuje kolumnę „Predictions” (3, 4, 1, 2). Strzałka „Evaluate” prowadzi od predykcji z powrotem do Model Buildera.]

17. Zbiór testowy (Test set)

- Kilka ważnych uwag o zbiorze testowym
 - Zbiór testowy trzeba pilnować starannie, także przed samym sobą, i używać go tylko oszczędnie.
 - Zbiór testowy powinien być trzymany osobno i prawie nigdy nie używany.
 - Nie wolno go używać do wyboru żadnego hiperparametru modelu (np. rzędu wielomianu (patrz dalej)).
 - Jeśli zostałby użyty do wyboru, przestałby być danymi niewidzianymi.
 - Używaj go wyłącznie do końcowej oceny.
 - W konkursach uczenia maszynowego zbiór testowy jest ukryty i prywatny.

18. Przykład

- Załóżmy, że zebraliśmy 3000 e-maili oznaczonych jako spam lub nie-spam.
 - $\mathbf{x}^{(i)}$ przechowuje liczbę wystąpień danego słowa w e-mailu.
 - Nasz zbiór docelowy $t = (0, 1)$, 0 oznacza nie-spam, 1 spam.
 - Zatem każde $y^{(i)}$ wynosi 0 lub 1.
 - Dzielimy zbiór na treningowy i testowy.
 - Dopasowujemy (*Fit*)
 - Oceniamy (*Evaluate*)
 - Co się stanie, jeśli mamy znacznie więcej e-maili nie-spam niż spam?
 - Potrzebujemy reprezentatywnej próby, tzn. określonego procentu e-maili spam, określonego procentu e-maili nie-spam jednego rodzaju, następnie innego procentu e-maili nie-spam innego rodzaju itd.

19. Rozpoznawanie przeuczenia / niedouczenia

- Często porównuje się wynik (*score*) na zbiorze treningowym z wynikiem na zbiorze testowym, aby sprawdzić, czy któreś z tych zjawisk występuje

```
lr.fit(X_train, y_train)

train_score = lr.score(X_train, y_train)
test_score = lr.score(X_test, y_test)
```

- Jeśli wynik treningowy jest bardzo słaby, model jest niedouczony
- Prawie zawsze wynik treningowy będzie lepszy niż testowy.
- Jeśli istnieje duża różnica między wynikiem treningowym a testowym (wynik treningowy znacznie lepszy), model jest przeuczony

20. Radzenie sobie z przeuczeniem / niedouczeniem

- To stały problem, ale istnieje kilka powszechnych technik.
- Przy niedouczeniu trzeba trenować dłużej lub użyć bardziej złożonego modelu. Może też być konieczne przyjrzenie się mechanizmom trenowania i funkcji straty. Czy są odpowiednie dla problemu?
- Przy przeuczeniu można:
 - Zmniejszyć złożoność modelu.
 - Zmniejszyć liczbę cech. Szczególnie gdy cech jest dużo w porównaniu z liczbą próbek treningowych. Ale tą metodą tracimy też informacje. Wybierz mądrze, co odrzucić.
 - Zdobyć więcej danych.
 - Najczęściej użyć regularyzacji (*regularisation*) (porozmawiamy o niej później).

21. Dobór złożoności modelu

- Przy stosunkowo prostych modelach (trenowanych tylko minuty-godziny) możemy sobie pozwolić na wypróbowanie wielu różnych rozmiarów modelu.
- Nie możemy jednak po prostu wybrać tego, który ma najniższy błąd treningowy – już pokazaliśmy, że może nas to wprowadzić w błąd.
- Zamiast tego potrzebujemy mechanizmu sprawdzania straty na danych, których algorytm trenujący nie widzi (zbiór testowy??).
- Technika, którą omówimy, nazywa się strojeniem hiperparametrów (*Hyper-parameter tuning*).

22. Hiperparametry (Hyper-parameters)

- **Definicja:**
 - Hiperparametry to parametry, które są wybierane, a nie uczone.
- Przykłady hiperparametrów:
 - stopień modelu, np. liniowy, kwadratowy, wielomian stopnia n. Które cechy użyć w modelu? (Czy każda informacja jest potrzebna?)
 - współczynnik uczenia (*learning rate*) α
 - siła regularyzacji (*Regularisation*)
 - liczba warstw/neuronów w sieci neuronowej

23. Dopasowanie krzywej wielomianowej (Polynomial curve fitting)

- Jaką postać ma $f(x)$?
 - Spróbujmy wielomianów stopnia M.

$$f(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_M x^M$$

- To jest przestrzeń hipotez (*hypothesis space*).
- Uwaga: $\mathbf{w}$ jest wektorem (dlatego pogrubione). $\mathbf{w} = (w_0, w_1, \dots, w_M)$
- Jak mierzymy sukces?
 - Suma kwadratów błędów (*Sum of squared errors*)

$$E(\mathbf{w}) = \frac{1}{2} \cdot \sum [f(\mathbf{x}_n, \mathbf{w}) - t_n]^2 \quad (\text{suma po } n = 1 \dots N)$$

- lub RMS

- Spośród funkcji z danej klasy wybieramy stopień, który minimalizuje ten błąd.
 - Stopień wielomianu jest **hiperparametrem**.
 - Naprawdę chcemy stopnia, który najlepiej oddaje trend w danych.

24. Który stopień wielomianu?

- Zielona linia to model „idealny”, czerwona linia to kształt zbudowanego modelu.
 - Jeśli $M = 9$, to $E(w^*) = 0$: to jest przeuczenie

[Cztery wykresy w układzie 2x2 (oś pozioma - x od 0 do 1, oś pionowa - t od -1 do 1). Na każdym: zielona sinusoida („idealny” model), niebieskie okręgi - 10 punktów danych z szumem wokół sinusoidy, czerwona linia - dopasowany wielomian. $M = 0$: pozioma prosta, nie oddaje trendu. $M = 1$: prosta malejąca, nadal nie oddaje kształtu. $M = 3$: gładka krzywa bardzo zbliżona do sinusoidy - dobre dopasowanie. $M = 9$: czerwona krzywa przechodzi przez każdy punkt, ale mocno oscyluje między nimi i gwałtownie wychyla się na krańcach - przeuczenie.]

25. Błąd RMS

$$f(x, w) = w_0 + w_1x + w_2x^2 + \dots + w_M x^M$$

- Oto wykres pierwiastka błędu średniokwadratowego (RMS) dla każdego M dla:
 - skuteczności zbioru treningowego względem modelu (informacje znane)
 - skuteczności zbioru testowego względem modelu (dane wcześniej niewidziane)

[Wykres liniowy: oś pozioma - M (0 do 9), oś pionowa - E_{RMS} (0 do 1). Niebieska linia „Training”: ok. 0,43 dla $M = 0$, ok. 0,32 dla $M = 1-2$, ok. 0,13 dla $M = 3-8$, spada do 0 przy $M = 9$. Czerwona linia „Test”: ok. 0,57 dla $M = 0$, ok. 0,43 dla $M = 1-2$, ok. 0,27 dla $M = 3-8$, a przy $M = 9$ gwałtownie rośnie do ok. 0,9.]

- Chodzi o to, że model radzi sobie bardzo, bardzo dobrze przy $M = 9$ na zbiorze treningowym. Radzi sobie bardzo, bardzo słabo przy $M = 9$ na zbiorze testowym, tzn. jest przeuczony przy $M = 9$.

26. Wybór hiperparametrów

- Potrzebujemy jakiejś metody wyboru hiperparametrów.
- Uwaga: istnieje wiele różnych metod wyboru hiperparametrów (nie jest to przedmiotem tego modułu, ale jedna metoda zostanie zademonstrowana). W przyszłości, jeśli ktoś zajmie się tym obszarem, będzie miał więcej do nauki.

27. Zbiór walidacyjny (Validation set)

- Zachowanie danych testowych osobno:
 - Podziel dane treningowe na zbiór treningowy i zbiór walidacyjny (często stosuje się podział 70/30).
 - Wytrenuj różne modele (np. wielomiany różnych rzędów) na zbiorze treningowym.
 - Przejdź przez wszystkie możliwe hiperparametry (np. wielomian $M=1$, potem $M=2$, potem $M=3$)
 1. Ustal hiperparametr (na początek $M=1$)
 2. Wytrenuj model na zbiorze treningowym
 3. Zwaliduj model na zbiorze walidacyjnym
 4. Zapisz wynik dla tego hiperparametru
 5. Powtórz od kroku 1 z następnym hiperparametrem.
 - Wybierz model (np. rząd wielomianu) z minimalnym błędem na zbiorze walidacyjnym.
 - (nie zawsze, ale często) Wytrenuj model ponownie na pełnych danych treningowych (zbiór treningowy + zbiór walidacyjny).

28. Błąd / strata testowa (Test error/loss)

- Teraz musimy być trochę ostrożni.
 - Hiperparametry to parametry modelu, które nie są trenowane, tzn. my je wybieramy.
 - Jak widzimy przy strojeniu hiperparametrów, nie trenujemy ich dokładnie, ale też nie wybieramy ich w pełni sami.
 - Strojenie w pewien sposób wykorzystuje dane ze zbioru walidacyjnego.

- Może nie widzieć tych danych bezpośrednio, ale dostaje z nich sygnał.
- W efekcie dane walidacyjne zostają „skompromitowane”.
- Nie dają nam prawdziwego obrazu generalizacji algorytmu.
- Do tego nadal potrzebujemy zbioru testowego, aby obliczyć błąd testowy.

29. Zbiór testowy (2)

- Zbiór testowy trzeba pilnować starannie, także przed samym sobą, i używać go tylko oszczędnie.
- Zbiór testowy powinien być trzymany osobno i prawie nigdy nie używany.
- Hiperparametry modelu wybieraj za pomocą zbioru walidacyjnego
- Końcowy model oceń zbiorem testowym.

30. Walidacja krzyżowa (Cross-validation)

- Użycie statycznego zbioru walidacyjnego może wprowadzić obciążenie (*Bias*). Co, jeśli zbiór walidacyjny nie jest reprezentatywny? Przy ograniczonej ilości danych jest to bardziej prawdopodobne.
- Idea walidacji krzyżowej (*Cross-Validation*) polega na wielokrotnych przebiegach walidacji
 - Walidacja krzyżowa tworzy S grup danych, $S - 1$ używa do trenowania, pozostałą do walidacji
 - Wykonaj kilka przebiegów z tymi różnymi grupami (zależnie od wybranej metody walidacji krzyżowej)
 - Uśrednij wyniki (błędy) dla różnych podziałów trening/walidacja
 - Wybierz hiperparametr o najniższym średnim błędzie ze wszystkich przebiegów

31. Walidacja krzyżowa (2)

- Walidacja krzyżowa jest skuteczną metodą doboru modelu, ale może być powolna
 - Modele z wieloma parametrami złożoności: wykładnicza liczba przebiegów
- Rodzaje walidacji krzyżowej
 - Walidacja z pominięciem jednej próbki (*Leave out one cross validation*, LOOCV)
 - Walidacja krzyżowa k -krotna (*k-fold cross validation*)
 - Stratyfikowana walidacja krzyżowa k -krotna (*Stratified k-fold cross validation*)
 - Walidacja adwersarialna (*Adversarial validation*)
 - Walidacja krzyżowa dla szeregów czasowych (*Cross validation for time series*)

[Diagram: cztery poziome paski (run 1 – run 4), każdy podzielony na cztery równe pola. W każdym przebiegu inne pole jest zaznaczone na różowo (zbiór walidacyjny): w run 1 pierwsze, w run 2 drugie, w run 3 trzecie, w run 4 czwarte; pozostałe pola są białe (zbiór treningowy).]

32. Walidacja krzyżowa k -krotna (K-fold cross validation)

- Losowo podziel cały zbiór danych na k „foldów” (części)
- Dla każdego k -tego foldu w zbiorze danych zbuduj/wytrenuj model na $k - 1$ foldach. Następnie oceń model, aby sprawdzić jego skuteczność na k -tym foldzie
- Zapisz błąd widoczny dla każdej z predykcji
- Powtarzaj, aż każdy z k foldów posłuży jako zbiór walidacyjny
- Średnia z k zapisanych błędów nazywana jest błędem walidacji krzyżowej i posłuży jako miara skuteczności modelu
- (5 i 10 to zwykle dobre wartości liczby foldów – domyślną w sklearn jest 5)

33. Walidacja krzyżowa 10-krotna (10-fold cross validation)

[Diagram: dziesięć pionowych szarych słupków obok siebie, ponumerowanych wierszami 1-10 po lewej stronie. W każdym słupku jedna dziesiąta wysokości jest zaznaczona na zielono („Validation”), a reszta jest szara („Training”). W pierwszym słupku zielony fragment jest na samej górze (wiersz 1), w drugim w wierszu 2, i tak dalej, aż do dziesiątego słupka, gdzie zielony fragment jest na samym dole (wiersz 10) – zielony pas przesuwają się po przekątnej.]

34. Czy to już ostateczna procedura?

- Zachowanie danych testowych osobno:
 - Przejdź przez wszystkie możliwe hiperparametry (np. wielomian $M=1$, potem $M=2$, potem $M=3$)
 1. Ustal hiperparametr (na początek $M=1$)
 2. Losowo podziel dane treningowe na k foldów
 - Wytrenuj model na $k - 1$ foldach zbioru danych.
 - Oceń model na pozostałym foldzie.
 - Zapisz błąd.
 - Powtarzaj, aż wszystkie foldy zostaną zbiorem walidacyjnym.
 3. Oblicz średni błąd ze wszystkich foldów i zapisz ten wynik dla danego hiperparametru
 4. Powtórz od kroku 1 z następnym hiperparametrem
 - Wybierz model (np. rząd wielomianu) z minimalnym (średnim) błędem na foldach
 - (nie zawsze, ale często) Wytrenuj model ponownie na pełnych danych treningowych (zbiór treningowy + zbiór walidacyjny).
- Model wybierasz za pomocą walidacji, a model końcowy możesz ocenić zbiorem testowym.

35. Procedura

1. Podziel swoje dane z etykietami/znanyymi wynikami na zbiór treningowy i zbiór testowy
2. Wybierz model bazowy (regresja liniowa, najbliżsi sąsiedzi itd.)
3. Dostrój hiperparametry modelu za pomocą walidacji krzyżowej
4. Wytrenuj model (z wybranymi parametrami) na zbiorze treningowym
5. Oceń model na zbiorze testowym

36. Regresja wielomianowa w Pythonie

- Twoją zmienną zależną jest y , a zmienną niezależną x . Masz zbiór danych X ze zbiorem odpowiedzi y . Chcemy modelu:

$$y = a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3 + \dots + a_mx^m$$

- Więc **przekształcamy** (*transform*) X do wybranego stopnia wielomianu.
- Wagi a_0 itd. są nadal liniowe, więc to nadal jest regresja liniowa!

```
In [9]: import numpy as np
In [2]: from sklearn.preprocessing import PolynomialFeatures
In [6]: x = np.array([1,2,3,4,5])
In [7]: X = PolynomialFeatures(3).fit_transform(x.reshape(-1,1))
In [8]: X
Out[8]: array([[ 1.,  1.,  1.,  1.],
               [ 1.,  2.,  4.,  8.],
               [ 1.,  3.,  9., 27.],
               [ 1.,  4., 16., 64.],
               [ 1.,  5., 25.,125.]])
```

- Zeszyt laboratoryjny (*lab-book*) na Moodle polega na wykonaniu k -krotnej walidacji krzyżowej z regresją wielomianową w celu wyboru stopnia wielomianu.